

Advanced topics in seasonal adjustment: JDemetra+ and R

Christiane Hofer, Rouven Stäcker / DG Statistics
5 November 2025

■ Agenda

- Introduction R and R Studio
- Introduction JD+ from R

Introduction to R and RStudio

What is R?

- Programming language for statistical computing and graphics
- Widely used tool for data analysis

What is RStudio?

- A user-friendly interface for programming in R
- IDE (Integrated development environment)

Cheat Sheet Base R

<https://github.com/rstudio/cheatsheets/blob/main/base-r.pdf>

Cheat Sheet RStudio

<https://github.com/rstudio/cheatsheets/blob/main/rstudio-ide.pdf>

Introduction to R and RStudio

Basic calculations

$1+2$

```
## [1] 3
```

$5*6$

```
## [1] 30
```

2^3

```
## [1] 8
```

$\log(6)$

```
## [1] 1.791759
```

Introduction to R and RStudio

Variables

```
a <- c(5, 3, 4)
```

```
b <- c(4, 6, 2)
```

```
a*b
```

```
## [1] 20 18 8
```

```
(a*b)/3
```

```
## [1] 6.666667 6.000000 2.666667
```

Introduction to R and RStudio

Lists

```
myList <- list(x = a, y = b, z = log(a))  
myList$x  
  
## [1] 5 3 4  
  
myList$y  
  
## [1] 4 6 2  
  
myList$z  
  
## [1] 1.609438 1.098612 1.386294
```

Introduction to R and RStudio

Time Series

```
data <- c(1:10)
myts <- ts(data, start = c(2023,1), frequency = 12)
myts
```

```
##           Jan Feb Mar Apr May Jun Jul Aug Sep Oct
## 2023      1   2   3   4   5   6   7   8   9  10
```

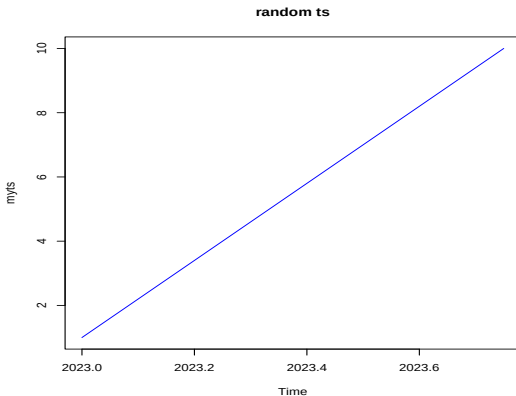
```
myts2 <- ts(data*2, start = c(2023,1), frequency = 12)
myts2
```

```
##           Jan Feb Mar Apr May Jun Jul Aug Sep Oct
## 2023      2   4   6   8  10  12  14  16  18  20
```

Introduction to R and RStudio

Display a Time Series

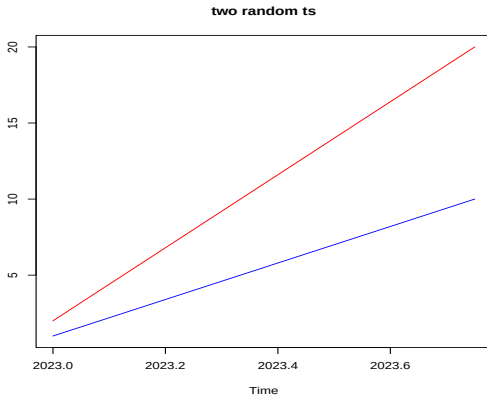
```
plot(myts, col = "blue", main = "random ts")
```



Introduction to R and RStudio

Creating a Time Series

```
ts.plot(myts, myts2, col = c("blue", "red"),  
        main = "two random ts")
```



Install packages with:

```
install.packages("readxl")  
install.packages("RJDemetra")
```

Activate the packages with:

```
library(RJDemetra)  
library(readxl)
```

Load data frame from Excel-File

```
data <- read_excel("TimeSeries.xlsx", sheet = "PKW")
```

1. Read the data frame "TimeSeries.xlsx" into R from Excel
2. Turn the data into a monthly time series starting in January 1990
3. Plot the time series and give the plot a fitting title
4. Multiply the time series by 100 and plot the result
5. Plot the logarithm of the time series

■ Agenda

- Introduction R and R Studio
- Introduction JD+ from R

JDemetra+ and R

Motivation

- Augment the developer community
- Exploring new algorithms/features
- Systematic testing
- Additional user interface, further processing

Solution

- Published R-Package (JDemetra+ Version 2.2.x)
- Explorative Code for R Packages (JD+ Version 3.x)

Package Repository

- CRAN

Requirements

- rJava ($\geq$ 0.9-8)
- Java SE 17 or higher

Issues and Code on Git Hub

- <https://github.com/rjdverse/rjdemetra/>

Documentation

- R embedded documentation

Authors

- Alain Quartier-la-Tente, Anna Smyk and others

R-Package (JD+ Version 2.2.x)

Example: Know nothing

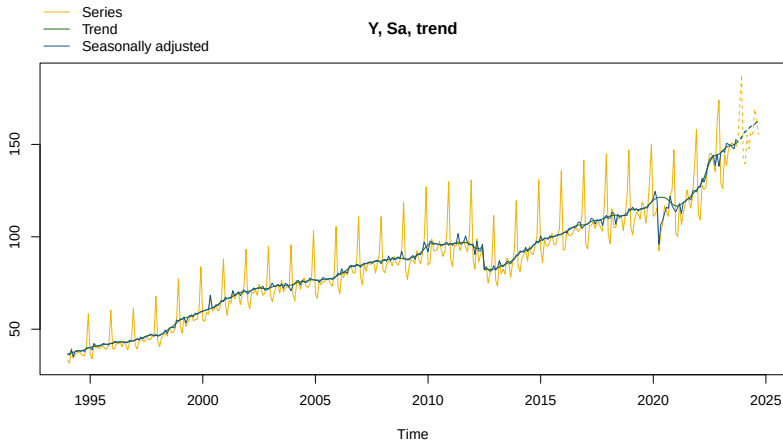
```
library(RJDemetra)
data <- readxl::read_excel("Trade.xlsx", sheet = 1)
myseries <- ts(data$cosmetics, start = c(1994, 1),
               frequency = 12)
x13_model <- x13(myseries) # X-13ARIMA method
#x13_model$decomposition$specification
#x13_model$regarima$regression.coefficients
#x13_model$final$series
x13_model$decomposition$mstats

##           M stats
## M(1)  0.10078917
## M(2)  0.04727197
## M(3)  0.50846243
## [ reached 'max' / getOption("max.print") -- omitted 10 r
```

R-Package (JD+ Version 2.2.x)

Example: Know nothing

```
plot(x13_model, type_chart = "sa-trend")
```



R-Package (JD+ Version 2.2.x)

Example: X11Spec

```
library(RJDemetra)
myspec1 <- x13_spec(spec = "X11", x11.mode="Additive")
my_model <- x13(myseries, spec = myspec1)
#my_model #summary output
```

```
my_model$final$series
```

```
##           y           sa           t           s           i
## Jan 1994  33.2  36.40958  36.75084  -3.20958343  -0.34125703
## Feb 1994  31.6  37.29474  36.99044  -5.69474138   0.30429768
## Mar 1994  39.7  40.11993  37.25882  -0.41993442   2.86110947
## Apr 1994  34.3  35.33433  37.56527  -1.03432731  -2.23093993
## [ reached 'max' / getOption("max.print") -- omitted 353 rows ]
```

R-Package (JD+ Version 2.2.x)

Example: Pre-specified outliers

```
myspec1 <- x13_spec(spec = "RSA3",  
                    usrdef.outliersEnabled = TRUE,  
                    usrdef.outliersType = c("LS"),  
                    usrdef.outliersDate = c("2020-04-01"))
```

```
out_model <- x13(myseries, myspec1)  
out_model$regarima$regression.coefficients #estimated
```

##		Estimate	Std. Error	T-stat
##	LS (4-2020)	-0.2073214	0.02312712	-8.964427
##	LS (7-2012)	-0.1484976	0.02079351	-7.141537
##	LS (6-2020)	0.1070746	0.02312977	4.629299

```
s_preOut(out_model)
```

##	type	date	coeff
## 1	LS	2020-04-01	NA

R-Package (JD+ Version 2.2.x)

Example: user-defined calendars

```
reg_data <- readxl::read_excel("Trade.xlsx", sheet = 2)
ts_reg<-ts(reg_data[,2], start = c(1950,1), frequency = 12)

myspec1 <- myspec1 <- x13_spec(spec = "RSA3",
                                tradingdays.option = "UserDefined",
                                usrdef.varType="Calendar",
                                usrdef.var=ts_reg,
                                usrdef.varEnabled = T,
                                estimate.from="1994-01-01")

cal_model <- x13(myseries, myspec1)
cal_model$regarima$regression.coefficients #estimated
```

##		Estimate	Std. Error	T-stat
##	userdef	-0.02905229	0.00249385	-11.649573
##	TC (4-2020)	-0.20973269	0.02241228	-9.357938
##	LS (7-2012)	-0.14857397	0.01994335	-7.449801
##	[reached 'max' / getOption("max.print") -- omitted 2 rows]			

R-Package (JD+ Version 2.2.x)

Example: Workspace in R I

```
dir <- tempdir() #directory of choice  
# Create and export a 'JDemetra+' workspace  
wk <- new_workspace()  
new_multiprocessing(wk, "sa1")  
# Add SA-Item  
add_sa_item(wk, "sa1", x13_model, "X13 SA-Item")  
add_sa_item(wk, "sa1", out_model, "X13 Outlier")  
  
#Save  
save_workspace(wk, file.path(dir, "workspace.xml"))
```

JDemetra+ UI can open, change and use workspace.xml

R-Package (JD+ Version 2.2.x)

Example: Workspace in R II

```
#Load WS
ws <- load_workspace( file.path(dir, "workspace.xml"))
#Calculate Ws
compute(ws)

# To extract first model
mp <- get_object(ws, 1) #Multiprocessing
sa <- get_object(mp, 1) #SA-Item
get_name(sa)

## [1] "X13 SA-Item"
```

Work without Workspace

- data is read completely from data source
- everything is estimated as defined in the specification

Work with Workspace

- no automatic refresh of data from data source
- workspace can be refreshed with UI, Cruncher, JDCruncher

R-Package (JD+ Version 2.2.x)

Example: use Java objects I

```
#jx13 returns the (jSA Object) Java objects of  
#the seasonal adjustment
```

```
myseries <- ipi_c_eu[, "FR"]  
mysa <- jx13(myseries, spec = "RSA5c")
```

```
get_dictionary(mysa)
```

```
## [1] "y"  
## [2] "y_f"  
## [3] "t"  
## [4] "t_f"  
## [5] "sa"  
## [6] "sa_f"  
## [7] "s"  
## [8] "s_f"  
## [ reached 'max' / getOption("max.print") -- omitted 326 entries ]
```

R-Package (JD+ Version 2.2.x)

Example: use Java objects II

#Results

```
get_indicators(mysa, "decomposition.d10")
```

```
## $decomposition.d10
```

##		Jan	Feb	Mar	Apr	May	Jun
## 1990		-3.23112424	-2.45358625	6.92606765	0.46855094	-2.53648128	8.58652688
## 1991		-3.18607624	-2.56877244	6.76211515	0.48073025	-2.43731681	8.78444653
## 1992		-2.96230026	-2.81860766	6.50479190	0.50513443	-2.44011384	9.10177488
## 1993		-2.87063337	-3.12899915	6.34172006	0.56849659	-2.62588295	9.58461516
## 1994		-2.90970212	-3.39113948	6.26254717	0.65764640	-2.95333714	10.02989689
## 1995		-3.15354010	-3.54121591	6.35424853	0.70143710	-3.24373983	10.43634302
##		Jul	Aug	Sep	Oct	Nov	Dec
## 1990		-0.93318080	-26.88627300	5.84425980	9.51277718	3.27525681	1.47198330
## 1991		-0.96321830	-26.88978248	5.80580137	9.40295915	3.15895332	1.71980899
## 1992		-0.95685315	-26.87859177	5.74926791	9.19709183	3.01106153	2.16973476
## 1993		-0.89739294	-26.95134964	5.71343361	9.01884480	2.82218126	2.63980962
## 1994		-0.72683798	-26.99605412	5.74298632	8.94861147	2.55444586	2.96552928
## 1995		-0.46190103	-27.25086279	5.82222095	9.03999618	2.35284825	3.01697689
##	[reached 'max' / getOption("max.print") -- omitted 25 rows]						

R-Package (JD+ Version 2.2.x)

Example: use Java objects II

To convert to the R object

```
mysa_R<-jSA2R(mysa)
```

```
mysa_R$final$series
```

##		y	sa	t	s	i
##	Jan 1990	92.1	92.84963	94.35045	-0.74963367	-1.500812957
##	Feb 1990	92.3	95.29209	94.51281	-2.99209296	0.779282254
##	Mar 1990	102.1	95.15793	94.46318	6.94207475	0.694740272
##	Apr 1990	93.0	94.46963	94.29456	-1.46962729	0.175072202
##	May 1990	93.3	93.86488	93.98596	-0.56487973	-0.121079065
##	Jun 1990	100.8	92.87984	93.60725	7.92015573	-0.727406058
##	Jul 1990	92.9	94.25620	93.20996	-1.35620068	1.046240284
##	Aug 1990	66.7	91.58556	92.90870	-24.88556134	-1.323138476
##	Sep 1990	95.8	93.39798	92.78022	2.40201820	0.617761719
##	Oct 1990	105.0	93.00573	92.76766	11.99426774	0.238075129
##	[reached 'max' / getOption("max.print") -- omitted 362 rows]					

Explorative Code for R Packages (JD+ Version 3.x)

<https://github.com/rjdemetra>

Packages	Content
<code>rjd3highfreq</code>	High-frequency routines : Fractional airline (with outliers detection) "SEATS", STL, X11
<code>rjdemetra3</code>	Several functions to interact with JDemetra+ v3.0 workspaces, incl. refresh
<code>rjd3tramoseats</code>	Options and outputs of TRAMO-SEATS
<code>rjd3x13</code>	Options and outputs of X-13
<code>rjd3sts</code>	Structural Time Series algorithms

The purpose and the quality of these packages is quite different

News around JDemetra+

News Letter

- Registration: anna.smyk@insee.fr

New Online Documentation

- <https://jdemetra-new-documentation.netlify.app/>

Blog

- <https://jdemetra-universe-blog.netlify.app/>

Github

- <https://github.com/rjdverse>